This document addresses frequently asked questions regarding eligibility, timelines, submissions, platform usage, credits, evaluation process, and participation guidelines. Shortlisted participants must review this alongside the Challenges Document and official instructions.

Q1. Can team leads apply or pitch solo without a team?

No. A minimum team of **2 members** is mandatory for pitching rounds. Max is 5 members

Q2. Are tech enthusiasts under 18 eligible for the hackathon?

The age limit for participants MUST 18 – 45 years.

Q3. What is the submission process and timeline?

Selected participants should have received the Challenges Document and a Google Form for Challenge Selection. Please read both thoroughly.

Important Dates:

May 15, 2026 Deadline to select and submit your challenge/idea. You need to select only 1 challenge and the deploy your solution

May 20, 2026 Final project submission deadline (Instructions for submission will be shared separately)

May 25–26, 2026 Virtual Regional Pitching Rounds

June 07, 2026 National Finale in Islamabad (Logistic support will be provided ONLY from Lahore/ Karachi to Islamabad)

From all submissions, **10–15 teams will be shortlisted** from each regions for virtual pitching. Regional winners/runners-up will qualify for the National Finale.

Q4. Can we use tools like n8n, Google AI Studio, Vertex AI, LangGraph, CrewAI, or external services with Antigravity?

Yes. As long as Antigravity remains the main orchestrator and development logs/ reasoning traces can be submitted. You are free to build agents in any framework (Vertex AI, LangGraph, etc.) and connect them via Antigravity.

Q5. Are the provided $5 credits enough for the hackathon?

Each team member wil get $5 credit. We can facilitate teams with **additional GCP credits** as required. These credits are intended for solution development on Google Cloud Platform services.

Q7. Can we use other LLMs inside Antigravity?

Yes. As long as your solution operates within the Antigravity environment, you may integrate other LLMs.

Q8. Is Mobile app mandatory?

Yes. Yes Mobile app is mandatory for all challenges. Web app is optional.

Q9. Do we have to submit the final solution on May 15?

No. May 15 is only for challenge/idea selection. The final project submission is due **May 20, 2026**.

Q10. Can we use our previous Phase #1 project in the Hackathon?

No, You will build your solution based on the explicit requirements of the selected challenge.

Q11. Can we make changes in the team composition?

Yes. You may make changes to your team composition during the development phase. However, you will be required to submit the finalized team details along with your final project submission.

Important Note: Please ask all questions within the official community channels or Discord. Queries will not be addressed via direct messages.

SHARED SUBMISSION CHECKLIST

- **Working prototype**: Mobile app is mandatory for all challenges. Web app/dashboard is optional unless needed for demonstration.
- **Demo video**: 3-5 minutes for Challenges 1-3; around 3 minutes for Challenge 4. Must show agentic workflow end to end.
- **Demo video:** 2–3-minute video which shows a screen recording on how your team made use of Antigravity solutions
- **Antigravity trace/logs**: Workplan, task plan, agent observations, reasoning, decisions, tool calls, action execution, error recovery, and final outcomes.
- **README/documentation**: Architecture, data schemas, tools/APIs, Antigravity role, setup steps, assumptions, privacy note, cost/latency, scalability, baseline comparison, and limitations.
- **Baseline comparison**: Show how agentic system performs better than simple heuristic or non-agentic implementation.
- **Robustness evidence**: At least one failure, edge case, contradiction, missing data, or fallback scenario demonstrated.
- **Cost and scalability note**: Cost per operation or API call estimate; 10x/100x scaling discussion; latency or throughput estimate.

Challenge 1: Autonomous Content-to-Action Agent (Insight → Action System)

CHALLENGE OVERVIEW

Organizations are flooded with reports, dashboards, news, policy updates, spreadsheets, customer feedback, and operational signals. Most AI systems stop at summarization. This challenge requires teams to build an agentic system that understands multi-source content, extracts meaningful insights, resolves conflicting evidence, decides what should be done, executes simulated action chains, and shows measurable outcome changes.

PROBLEM STATEMENT

- Ingest multiple content sources at the same time, including unstructured, semi-structured, and structured inputs.
- Extract meaningful insights, trends, risks, contradictions, and opportunities.
- Analyze implications under real-world constraints such as cost, time, resources, urgency, and API limits.
- Generate and prioritize 3-5 interconnected actions instead of a single action.
- Simulate execution of the action chain with visible state changes after every step.
- Recover from partial failures, conflicting evidence, or missing data through fallback or clarification actions.

MANDATORY REQUIREMENT: GOOGLE ANTIGRAVITY

- Use Google Antigravity as the core platform for agent orchestration, reasoning, planning, tool/API integration, and action execution.
- Use Antigravity logs to show the workplan, task plan, decision trace, tool calls, action execution, and recovery steps.
- External LLMs, APIs, databases, dashboards, and visualization tools are allowed, but Antigravity must remain central to the system logic.

SYSTEM REQUIREMENTS

- **Advanced content understanding**: Process at least five input items across multiple types: PDF/report, website/article, CSV/JSON, table/dashboard, and mock real-time feed.
- **Temporal analysis**: Detect how key signals change over time, such as sales decline, inventory shortage, complaint spike, price change, or public sentiment shift.
- **Contradiction detection**: Identify conflicting claims across sources, score source credibility and recency, and propose resolution actions.

- **Noise filtering**: Separate real signals from irrelevant, duplicate, spam-like, stale, or low-credibility content.
- **Insight resolution**: When data conflicts, explain the conflict and generate an investigation path instead of forcing a false conclusion.
- **Multi-step action chain**: Each scenario must trigger 3-5 connected actions, such as diagnose cause -> notify stakeholder -> update system -> launch mitigation -> schedule follow-up monitoring.
- **Constraint-based decision-making**: Attach budget, time, resource, urgency, and rate-limit constraints to actions. The system must choose feasible actions and reject/modify infeasible ones.
- **Failure recovery and rollback**: Show what happens if one action fails, such as API failure, invalid data, notification failure, or rejected update. Recover, retry, or roll back state.
- **Outcome visualization**: Show before vs after state, action logs, cost/latency metrics, and projected impact.

EXAMPLE SCENARIO

Input: Five sources indicate a possible inventory shortage: warehouse spreadsheet, supplier email, sales dashboard, customer complaints, and news about transport delays. One source says stock is sufficient while another says stock will run out in two days.

- **Insight:** Demand is rising while supplier reliability is falling; one source is stale and contradicts newer data.
- **Resolution logic:** Check timestamp, credibility, and affected SKUs; mark the older warehouse sheet as stale.
- **Action chain:** 1) validate stock, 2) notify procurement, 3) simulate emergency order, 4) update customer delivery estimates, 5) schedule 24-hour monitoring.
- **Constraints:** Emergency order budget limit PKR equivalent, supplier lead-time limit, and notification deadline.
- **Outcome:** Dashboard shows reduced stockout risk, updated supplier status, customer notification draft, and cost/latency summary.

RECOMMENDED STRESS-TEST SCENARIOS

- Same metric appears in three sources with conflicting values.
- Recommended action violates budget or deadline constraints.
- One action in the chain fails and must be retried, replaced, or rolled back.
- A low-credibility source creates a false signal that the system must down-rank.
- One action creates a side effect in another business area, requiring what-if analysis.

DELIVERABLES

- Working prototype with a mobile app as mandatory and web app as optional.
- Demo video of 3-5 minutes showing input -> insight -> contradiction or constraint handling -> action chain -> simulation -> outcome.
- Antigravity agent trace/logs showing workplan, task plan, reasoning steps, tool calls, decisions, failures, and recovery.
- README with architecture, data sources, tools/APIs, Antigravity role, assumptions, constraints, cost/latency analysis, baseline comparison, and limitations.

EVALUATION CRITERIA

- Antigravity integration 20%
- Agentic reasoning and workflow 20%
- Insight quality and contradiction handling 20%
- Action chain and outcome simulation 15%
- Robustness, scalability, cost and latency 15%
- Innovation and UX 10%

Challenge 2: AI Service Orchestrator for Informal Economy

CHALLENGE OVERVIEW

Informal service economy includes plumbers, electricians, tutors, beauticians, drivers, mechanics, AC technicians, home service providers, and small local professionals. Service discovery often happens through WhatsApp, phone calls, referrals, and informal networks, causing missed opportunities, poor matching, unpredictable pricing, weak follow-up, and limited trust. This challenge requires an agentic system that automates the end-to-end service lifecycle from natural-language request to matching, scheduling, pricing, booking, follow-up, quality feedback, and dispute handling.

PROBLEM STATEMENT

1. Understand multilingual service requests (for example Urdu, Roman Urdu, English, and mixed/code-switched language).
2. Extract service type, location, urgency, preferred time, constraints, and user preferences.
3. Discover providers using mock data, Google Maps/Places, or other APIs.
4. Rank providers using a multi-factor matching algorithm rather than distance alone.
5. Generate dynamic price quotes with transparent breakdowns.
6. Simulate booking, confirmation, reminders, service progress, feedback, reputation updates, and dispute resolution.

MANDATORY REQUIREMENT: GOOGLE ANTIGRAVITY

- Use Google Antigravity as the main orchestrator for intent understanding, matching, scheduling, pricing, booking, follow-up, and service-quality workflows.
- Show Antigravity reasoning traces for provider selection, price estimation, scheduling conflicts, confirmation actions, and dispute escalation. External LLMs, Maps/Places APIs, spreadsheets, mock datasets, and messaging APIs may be used, but Antigravity must control the agentic workflow.

SYSTEM REQUIREMENTS

- **Multilingual and noisy input handling**: Handle Urdu, Roman Urdu, English, misspellings, slang, and code-switching such as "Mujhe kal morning main AC service chahiye". Provide confidence score and confirmation questions when confidence is low.
- **Advanced provider matching**: Use at least six factors: distance/travel time, availability, rating, review recency, reliability/on-time score, skill specialization, price, capacity, cancellation rate, user preference, and risk score.

- **Skill and job complexity classification**: Classify job complexity as basic, intermediate, or complex and match to provider specialization, experience, tools, or certifications.
- **Scheduling intelligence**: Prevent double booking, include travel-time buffers, suggest alternate slots, manage waitlists, and reschedule automatically if a provider cancels.
- **Dynamic pricing**: Calculate price using demand, urgency, distance, service complexity, provider rate, loyalty discount, and surge conditions. Show breakdown and fairness to both user and provider.
- **Booking simulation**: Simulate booking confirmation, provider assignment, calendar update, SMS/WhatsApp notification, receipt, and database/spreadsheet update.
- **Service-quality loop**: Simulate en-route update, service completion checklist, photo/video evidence placeholder, customer feedback, rating adjustment, and future matching impact.
- **Dispute and escalation workflow**: Handle no-show, cancellation, quality complaint, price disagreement, overrun, refund, compensation, blacklist, or human escalation simulation.
- **Provider-side optimization**: Show provider workload balancing, fair earning opportunity, demand forecasting, or recommended time slots for provider utilization.
- **Robustness and fallback**: Handle no provider available, low-confidence language parsing, Maps/API failure, payment confirmation failure, and user preference conflicts.

EXAMPLE SCENARIO

- **User input**: "AC bilkul kaam nahi kar raha, kal subah G-13 mein technician chahiye, budget zyada nahi hai."
- **Understanding**: Service = AC repair; issue severity = high; location = G-13; time = tomorrow morning; price sensitivity = high.
- **Matching**: Ranks providers using travel time, availability, AC specialization, on-time score, review sentiment, rate, cancellation risk, and capacity.
- **Decision**: Recommends Provider A despite Provider B being closer because Provider A has higher reliability and specialized AC repair reviews.
- **Pricing**: Generates quote with visit fee, distance cost, urgency adjustment, and budget-sensitive alternative.
- **Simulation**: Books 10:00 AM slot, sends confirmation, updates booking sheet, schedules reminder, simulates provider en-route update, collects feedback, and updates reputation score.

RECOMMENDED STRESS-TEST SCENARIOS

- No suitable provider is available in the requested time window.
- A provider cancels after confirmation and the system must reschedule.
- User input is misspelled, mixed-language, or ambiguous.
- Two users request the same provider at overlapping times.
- Customer disputes price or quality after service completion. Provider has high rating but recent negative reviews and high cancellation rate.

DELIVERABLES

1. Working prototype with a mobile app as mandatory and web app as optional.
2. Demo video of 3-5 minutes showing user request -> intent extraction -> provider ranking -> pricing -> booking -> follow-up -> feedback/dispute workflow.
3. Antigravity agent trace/logs showing language parsing confidence, provider ranking rationale, scheduling decisions, price logic, action execution, and fallback behavior.
4. README with architecture, provider dataset schema, matching factors, Antigravity workflow, APIs/tools, assumptions, cost/latency analysis, baseline comparison, privacy note, and limitations.

EVALUATION CRITERIA

- Antigravity integration 20%
- Matching and decision quality 25%
- Multilingual robustness and edge cases 15%
- Scheduling, pricing and service workflow 15%
- Dispute handling, reliability and scalability 15%
- Innovation and UX 10%

Challenge 3: Crisis Intelligence & Response Orchestrator (CIRO)

CHALLENGE OVERVIEW

Cities frequently face localized crises such as urban flooding, heatwaves, road blockages, accidents, infrastructure failures, public disorder, disease spikes, and power outages. Signals may exist across social media, traffic maps, weather alerts, citizen complaints, emergency calls, sensors, and field reports, but response systems are often fragmented and reactive. This challenge requires an agentic system that fuses signals, detects emerging crises, predicts severity, allocates resources, coordinates stakeholders, simulates response actions, and recovers from false alarms or missed detections.

PROBLEM STATEMENT

1. Ingest and fuse at least three signal sources, such as social posts, weather, traffic, emergency calls, mock sensors, field reports, and historical data.
2. Detect and classify crisis type, location, severity, confidence, affected population, expected duration, and likely evolution.
3. Prioritize and allocate constrained response resources across one or more simultaneous crises.
4. Simulate coordinated actions such as traffic rerouting, emergency dispatch, hospital preparation, utility escalation, and public alerts.
5. Predict outcomes, side effects, and unintended consequences for each response action. Handle false positives, false negatives, and conflicting signals through verification and escalation logic.

MANDATORY REQUIREMENT: GOOGLE ANTIGRAVITY

- Use Google Antigravity to orchestrate multi-agent crisis detection, signal fusion, severity analysis, resource allocation, stakeholder communication, and action simulation.
- Show Antigravity traces for signal interpretation, confidence scoring, priority ranking, resource trade-offs, action execution, and recovery from false or conflicting signals.
- External APIs and mock streams are allowed, but Antigravity must coordinate planning and execution.

ENHANCED SYSTEM REQUIREMENTS

- **Multi-signal fusion**: Use at least three sources: social media/citizen posts, weather, maps/traffic, emergency call frequency, mock sensors, public transport, or historical vulnerability maps.

- **Source credibility and misinformation handling**: Score source credibility, geolocation confidence, urgency language, mention velocity, and contradiction level. Flag low-confidence or suspicious signals.
- **Crisis classification**: Classify type such as flood, heatwave, accident, infrastructure, power outage, protest, or disease cluster; include severity level and confidence score.
- **Severity and evolution prediction**: Estimate affected radius, population, duration, peak impact time, spread risk, and uncertainty range.
- **Resource allocation optimization**: Model constrained resources such as ambulances, police units, rescue teams, shelters, generators, water tankers, field teams, or drones. Allocate based on impact, urgency, travel time, and resource availability.
- **Multi-crisis coordination**: Handle at least two simultaneous incidents and show trade-offs in prioritization and resource assignment.
- **Impact simulation**: For each action, show before state, response action, expected after state, response time improvement, congestion impact, resource cost, and possible side effects.
- **Stakeholder notification**: Generate tailored messages for the public, emergency services, hospitals, utility companies, transport authority, and media/command center.
- **False positive/negative handling**: Simulate a false alarm, early low-confidence signal, or conflicting signals and show verification, escalation, correction, or alert retraction.
- **Robustness and degraded mode**: Handle API downtime, stale data, missing location, duplicate incidents, and rate limits with fallback sources or manual escalation.

EXAMPLE SCENARIO

Input signals: Social posts report flooding in G-10, weather API shows heavy rainfall, traffic API shows congestion spike, and one field report suggests a broken water main instead of flooding. At the same time, a heat emergency is reported in a nearby low-income neighborhood.

- **Detection**: Classifies G-10 incident as probable urban flooding with confidence score and identifies conflicting water-main hypothesis.
- **Prediction**: Estimates affected zones, likely duration, congestion spread, and vulnerable population risk.
- **Resource allocation:** Prioritizes rescue teams and police traffic units for G-10 while assigning medical outreach to heat emergency based on severity and resource constraints.
- **Simulation**: Reroutes traffic, creates emergency ticket, sends public alert, notifies hospital, updates incident dashboard, and simulates impact on response time and congestion.
- **Recovery**: If field verification confirms only a water-main burst, system updates classification, retracts public flood alert, and notifies utility provider.

RECOMMENDED STRESS-TEST SCENARIOS

- Two or more crises occur within 30 minutes and compete for limited emergency resources.
- Social media indicates flooding but official sensor data is unavailable or contradictory.
- An API fails mid-response and the system must use cached or alternate data.
- Public alert causes evacuation congestion, requiring staged alerting or rerouting. False alarm requires correction, apology/retraction, and log update.

DELIVERABLES

1. Working prototype with a mobile app as mandatory and web app/dashboard as optional.
2. Demo video of 3-5 minutes showing multi-source input -> crisis detection -> severity prediction -> resource allocation -> simulated response -> impact visualization -> recovery scenario.
3. Antigravity agent trace/logs showing signal fusion, confidence scoring, crisis classification, allocation trade-offs, stakeholder messages, action execution, and fallback behavior.
4. README with architecture, data stream schemas, Antigravity usage, APIs/tools, assumptions, privacy/safety note, cost/latency analysis, baseline comparison, scalability discussion, and limitations.

EVALUATION CRITERIA

- Antigravity integration 20%
- Crisis detection and severity analysis 25%
- Resource optimization and multi-crisis coordination 20%
- Impact simulation and stakeholder coordination 15%
- Robustness, scalability, cost and latency 10%
- Innovation and UX 10%

Challenge 4: The Mobile App Alchemy: Agentic Game Quest *(Genre-Agnostic: Hyper-Casual / RPG / Puzzle / Strategy/ Single/ Multiplayer)*

CHALLENGE OVERVIEW

Mobile gaming is evolving beyond static levels and predictable mechanics. This challenge asks teams to build a mobile game in any genre - hyper-casual, puzzle, RPG, strategy, simulation, multiplayer, or hybrid - where Google Antigravity drives intelligent game behavior. The game must show that agentic AI improves gameplay by adapting difficulty, generating content, learning from player behavior, coordinating NPCs/opponents, and optimizing engagement.

PROBLEM STATEMENT

1. Build a working mobile game with a clear core loop: player action -> feedback -> reward -> progression.
2. Use Google Antigravity to drive agentic gameplay decisions, not merely decorative AI responses.
3. Show how the agent observes player behavior, infers skill or strategy, decides changes, executes game adaptations, and evaluates outcome.
4. Implement adaptive difficulty using multiple player-performance and engagement signals.
5. Include agent traces that explain why a level, NPC behavior, reward, hint, or challenge was generated.
6. Compare the agentic version against a non-agentic baseline or fixed-rule version.

MANDATORY REQUIREMENT: GOOGLE ANTIGRAVITY

- Use Google Antigravity as the primary orchestrator for gameplay logic, NPC/opponent reasoning, procedural content generation, difficulty adaptation, rewards, validation, or narrative decisions.
- Antigravity must manage structured agent workflows and state decisions such as skill assessment, level generation, fairness validation, and reward logic.
- Teams must show Antigravity decision traces: what was observed, inferred, decided, executed, and measured.

WHAT COUNTS AS AGENTIC?

1. Not Sufficient

- Fixed rule: if score > 1000, set difficulty to hard.
- Preloaded level_7.json is selected from a fixed library.

- NPC attacks when player is near.
- Random rewards or fixed loot table.

2. Expected Agentic Behavior

- Agent observes score, timing, accuracy, retry behavior, session trend, and strategy pattern; infers skill and boredom/frustration risk; generates a difficulty adjustment with explanation.
- Agent generates a new level configuration for target difficulty, checks solvability/fairness, rejects poor configurations, and stores reasoning trace.
- NPC observes the player's dominant strategy, predicts next move, changes tactics, and adapts again if the player counters.
- Reward agent evaluates effort, progress, retention risk, and fairness before assigning a reward or challenge.

SYSTEM REQUIREMENTS

- **Core loop engine**: Game must have a polished, playable, repeatable loop with smooth feedback, scoring, rewards, and progression.
- **Adaptive difficulty**: Use at least five factors such as completion time, accuracy, resource efficiency, retry count, win/loss ratio, session length, quit behavior, learning speed, or strategy consistency.
- **Intelligent NPC/opponent behavior**: NPCs or opponents should observe, reason, adapt, and remember patterns. If genre has no NPCs, use an equivalent challenge-generation or puzzle-master agent.
- **Procedural content generation**: Agent generates levels, quests, puzzles, encounters, or missions; validates solvability, fairness, target difficulty, and novelty.
- **Quality-control agent**: Validate generated content to avoid impossible, trivial, unfair, or broken levels. Show acceptance/rejection logs.
- **Retention and engagement metrics**: Track session length, retries, churn risk, difficulty spike abandonment, challenge acceptance, feature exploration, and satisfaction proxy.
- **Fair play/referee logic**: Agent checks rule compliance, scoring validity, reward fairness, anti-exploit behavior, and multiplayer fairness where applicable.
- **Comparative baseline**: Show agentic vs fixed-rule gameplay comparison using engagement, win rate, retry rate, or player satisfaction proxy.
- **Optional multimodal innovation**: Camera, microphone, location, or multiplayer features may be used if privacy-preserving and relevant to gameplay.
- **Privacy and safety**: Do not store raw personal images/audio/location. Use on-device processing, anonymized signals, or explicit mock data where needed.

EXAMPLE SCENARIO

A puzzle/adventure game tracks that the player completes Level 5 in 45 seconds against a 60-second par time with 98% accuracy and two sessions of steady improvement. The agent infers that the player may become bored if difficulty remains unchanged.

- **Observation**: Fast completion, high accuracy, low retry count, increasing session length.
- **Inference**: Player is advanced; current puzzle family is too easy.
- **Decision**: Increase difficulty by 1.3x and introduce one new mechanic while keeping expected win rate around 50-60%.
- **Action**: Generate new level with additional obstacle pattern, validate solvability, and adjust reward.
- **Evaluation**: Track whether the next attempt creates a close-call moment without causing frustration.

RECOMMENDED STRESS-TEST SCENARIOS

- Player repeatedly fails and the agent must reduce frustration without making the game boring.
- Player discovers an exploit or dominant strategy; NPC or level agent must counter it fairly.
- Generated level is impossible and quality-control agent must reject and regenerate it.
- Agentic version must demonstrate better engagement or balanced difficulty than fixed-rule baseline.
- Multiplayer matchmaking must avoid unfair pairings or repeated mismatches if multiplayer is implemented.

DELIVERABLES

- Working mobile game prototype.
- Demo video of around 3 minutes focusing on gameplay and showing how Antigravity makes the experience adaptive, intelligent, and replayable.
- Architecture map showing Antigravity workflows, agent roles, state management, player metrics, and tool/data flow.
- Agent trace/logs showing observation, inference, decision, action, and outcome evaluation.
- README explaining the game hook, baseline comparison, engagement metrics, content-generation logic, privacy approach, and limitations.

EVALUATION CRITERIA

- Antigravity integration 25%
- Gameplay engagement and retention 25%
- Agentic innovation 20%
- Technical polish 15%

- Originality and creativity 10%
- Comparative proof bonus +5%